

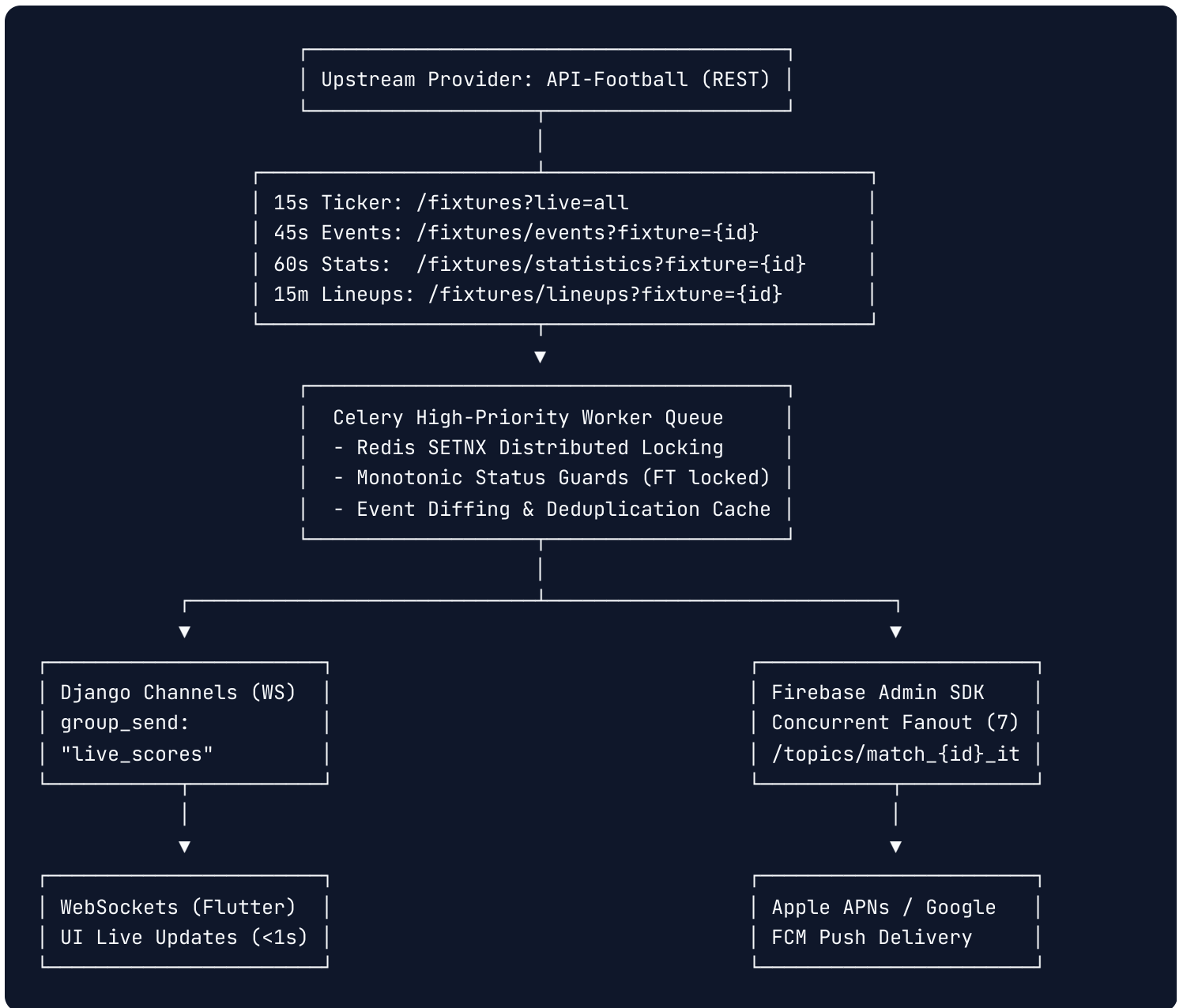
PRODUCTION VERIFIED SPECIFICATION

ScoreLivePRO: Backend Architecture & Notification Engine Specification

DOCUMENT VERSION 2.4.1 (Production Standard)	RELEASE DATE September 2026	TARGET STACK Django 4.x / Python 3.12, Celery 5.x, Redis 7
PUSH GATEWAY & CLIENT Google Firebase FCM v1 Topics / Flutter Client		

1. Executive System Overview

The ScoreLivePRO backend functions as a high-throughput, real-time sports synchronization and push notification engine. It ingests live football data from commercial REST APIs (API-Football / API-Sports), normalizes statuses across 16 FIFA states, broadcasts live score streams over WebSockets via Django Channels, and dispatches localized push notifications to iOS and Android devices via Google Firebase Cloud Messaging (FCM).



2. Real-Time Task Architecture & Celery Beat Schedule

All live data ingestion is managed asynchronously by Celery workers decoupled from HTTP request-response cycles. Tasks are partitioned into two priority queues:

- **high_priority** : Live tickers, events, statistics, lineups, kickoff reminders, and scheduled push alerts.
- **celery** (Default): Maintenance, standings, H2H warmup, daily league schedules, and background reconciliation.

2.1. Task Routing Configuration (config/celery.py)

```
app.conf.task_default_queue = 'celery'
app.conf.task_routes = {
    # High-Priority Real-Time Tasks & Notifications
    'sports.tasks.update_live_fixtures': {'queue': 'high_priority'},
    'sports.tasks.fetch_live_events': {'queue': 'high_priority'},
    'sports.tasks.fetch_live_statistics': {'queue': 'high_priority'},
    'sports.tasks.fetch_lineups_near_kickoff': {'queue': 'high_priority'},
    'sports.tasks.check_upcoming_matches_and_notify': {'queue': 'high_priority'},
    'sports.tasks.process_scheduled_notifications': {'queue': 'high_priority'},
    'notifications.tasks.*': {'queue': 'high_priority'},

    # Standard Maintenance & Background Sync Tasks
    'sports.tasks.fetch_upcoming_fixtures': {'queue': 'celery'},
    'sports.tasks.daily_maintenance_workflow': {'queue': 'celery'},
    'sports.tasks.warmup_upcoming_h2h': {'queue': 'celery'},
    'sports.tasks.fetch_standings_hourly': {'queue': 'celery'},
    'sports.tasks.cleanup_stale_live_fixtures': {'queue': 'celery'},
    'sports.tasks.reconcile_stuck_suspended_fixtures': {'queue': 'celery'},
    'sports.tasks.fetch_teams_for_active_leagues': {'queue': 'celery'},
    'sports.tasks.notify_daily_league_schedule': {'queue': 'celery'},
}
```

2.2. Celery Beat Schedules

Task Name	Interval	Queue	Description
update_live_fixtures	15.0s	high_priority	Polls /fixtures?live=all , saves match state, triggers goal/whistle alerts, and broadcasts to Channel Layer.
fetch_live_events	45.0s	high_priority	Polls /fixtures/events for matches with followers, diffs cards/subs/VAR, and triggers push alerts.
fetch_live_statistics	60.0s	high_priority	Refreshes possession, shots, corners, and fouls for active live fixtures.
fetch_lineups_near_kickoff	15 mins	high_priority	Polls confirmed starting XI lineups 45–60 min before kickoff; notifies followers.
process_scheduled_notifications	60.0s	high_priority	Dispatches admin-scheduled marketing or custom alerts.
check_upcoming_matches_and_notify	Every 15 mins	high_priority	Sends pre-match reminders 15 minutes before scheduled kickoff.
daily_maintenance_workflow	03:00 AM Daily	celery	Sequential update chain: Countries → Seasons → Leagues → Teams → Schedule.
warmup_upcoming_h2h	04:30 AM Daily	celery	Warms up Head-to-Head cache for all matches scheduled in the next 48 hours.
notify_daily_league_schedule	08:00 AM Daily	celery	Dispatches daily match counts and schedules for followed leagues.
fetch_standings_hourly	Hourly (:00)	celery	Refreshes league standings and points tables.

Task Name	Interval	Queue	Description
cleanup_stale_live_fixtures	Every 30 mins	celery	Reconciles zombie fixtures stuck in live status for > 4 hours.
reconcile_stuck_suspended_fixtures	Every 10 mins	celery	Verifies rescheduled/suspended fixtures against upstream calendar.

3. Concurrency Protection & Monotonic Integrity Guards

3.1. Distributed Redis Locking (SETNX)

To prevent worker concurrency races or duplicate API polling under high event volume, every critical task is protected by atomic Redis locks:

```
def acquire_lock(lock_id, expire=60):
    try:
        r = get_redis_client()
        # Atomic SET with NX=True ensures only one worker succeeds
        return bool(r.set(lock_id, "LOCKED", nx=True, ex=expire))
    except Exception as e:
        print(f"⚠️ REDIS LOCK ERROR: {e}")
        return True # Fail open to prevent permanent worker stall
```

- task-lock-live-updates : Expires in **15 seconds** (ticker loop).
- task-lock-live-events : Expires in **40 seconds** (detailed events).
- task-lock-kickoff-lineups : Expires in **300 seconds** (lineup polling).
- task-lock-stats-update : Expires in **60 seconds** (live statistics).
- task-lock-standings-hourly : Expires in **600 seconds** (standings).

3.2. Monotonic Status Guard (Prevents Post-Match Regressions)

A recurring problem in commercial sports APIs is that finished matches (FT , AET , PEN , ABD , AWD , WO) occasionally get temporarily omitted from live feeds or reported with stale in-play statuses (2H , HT) during API server caching hiccups.

```
# --- MONOTONIC STATUS GUARD ---
# A finished match (FT, AET, PEN, ABD, AWD, WO) must never regress to an in-play (2H, 1H, HT) or
if fixture_exists and old_status in Fixture.FINISHED_STATUSES:
    if new_status not in Fixture.FINISHED_STATUSES:
        print(f"🛡️ [MONOTONIC GUARD] Prevented status regression for Match {fixture_id}: attempt
        new_status = old_status
        status_data['short'] = old_status
        status_data['long'] = existing_fixture.status_long
        status_data['elapsed'] = existing_fixture.elapsed or 90
        status_data['extra'] = existing_fixture.extra
        g = existing_fixture.goals or g
        s = existing_fixture.score or s
```

4. Push Notification Engine Architecture

4.1. 5-Tier Hierarchical Subscription Model

Notifications are sent via Google Firebase Topics rather than mass device token loops. This guarantees sub-second dispatch regardless of whether 100 or 1,000,000 users follow a match.

Subscription Layer	Topic Format	Fanned-Out By	Description
1. Match Layer	/topics/match_{fixture_id}_{lang}	7 Languages	Users who individually starred the match.
2. Home Team Layer	/topics/team_{home_id}_{lang}	7 Languages	Users who follow the home club.
3. Away Team Layer	/topics/team_{away_id}_{lang}	7 Languages	Users who follow the away club.
4. League Layer	/topics/league_{league_id}_{lang}	7 Languages	Users who follow the entire tournament/league.
5. Global Layer	/topics/global_{lang}	7 Languages	Broad announcements and global app notifications.
Direct Inbox Layer	/topics/user_{user_id}	User Profile Locale	Account alerts, security, and direct messages.

Why Users Receive Alerts for Unstarred Matches:

If a user favorited *River Plate* or the *Argentine Primera División*, they are subscribed to `team_435_it` and `league_128_it`. When River Plate plays Huracán, the backend dispatches to team and league topics as well as the match topic. In the notification payload, the mobile app receives: `data["reason"] = "Following River Plate" or "Following Argentine Primera"`.

4.2. Multi-Language Concurrent Fanout

The engine supports 7 official languages: `en`, `es`, `fr`, `de`, `it`, `pt`, and `tr`. Alerts are fanned out concurrently using Python's `ThreadPoolExecutor`:

```
from concurrent.futures import ThreadPoolExecutor

def _send_lang(lang):
    t_title, t_body = translate_notification(title, body, event_type, lang)
    return NotificationService.send_push_to_topic(
        f"{topic}_{lang}", t_title, t_body, data, event_type, is_internal=True
    )

langs = ['en', 'es', 'fr', 'de', 'it', 'pt', 'tr']
with ThreadPoolExecutor(max_workers=len(langs)) as executor:
    list(executor.map(_send_lang, langs))
```

4.3. Complete Match Event Mapping & Triggers

Event Type	Trigger Condition	Service Method	Payload event_type
 Goal	<code>new_goals > old_goals</code>	<code>send_goal_alert</code>	GOAL
 Disallowed Goal	<code>new_goals < old_goals</code> (Score decreased)	<code>send_disallowed_goal_alert</code>	DISALLOWED_GOAL
 Yellow Card	New <code>Card</code> with 'Yellow Card'	<code>send_card_alert</code>	CARD
 Red Card	New <code>Card</code> with 'Red Card'	<code>send_card_alert</code>	CARD
 Substitution	New <code>subst</code> in <code>events[]</code>	<code>send_substitution_alert</code>	SUBSTITUTION
 VAR Review	New <code>Var</code> in <code>events[]</code>	<code>send_var_alert</code>	VAR
 Kickoff Whistle (1H)	<code>old_status</code> in <code>['NS','TBD']</code> & <code>new_status</code> in <code>['1H','LIVE']</code>	<code>send_kickoff_alert</code>	KICKOFF
 Half-Time (HT)	<code>old_status</code> $\neq$ 'HT' & <code>new_status</code> = 'HT'	<code>send_half_time_alert</code>	HALF_TIME
 2nd Half Kickoff (2H)	<code>old_status</code> = 'HT' & <code>new_status</code> = '2H'	<code>send_second_half_alert</code>	SECOND_HALF
 Extra Time (ET)	<code>old_status</code> in <code>['2H','FT','BT']</code> & <code>new_status</code> = 'ET'	<code>send_extra_time_alert</code>	EXTRA_TIME
 Penalty Shootout (P)	<code>old_status</code> in <code>['ET','BT']</code> & <code>new_status</code> = 'P'	<code>send_penalty_shootout_alert</code>	PENALTY_SHOOTOUT

Event Type	Trigger Condition	Service Method	Payload event_type
 Missed Penalty	New Goal with 'Missed Penalty'	send_missed_penalty_alert	MISSED_PENALTY
 Own Goal	New Goal with 'Own Goal'	send_own_goal_alert	OWN_GOAL
 Match Finished (FT)	old_status $\neq$ 'FT' & new_status in ['FT', 'AET', 'PEN']	send_match_result_alert	FULL_TIME
 Match Disruption	Status changes to PST , SUSP , INT , ABD , CANC	send_match_disruption_alert	POSTPONED , etc.
 Official Lineups	New starting XI saved in FixtureLineup	send_lineup_alert	LINEUPS
 Kickoff Reminder	Match begins in 15 mins (scheduled)	send_match_reminder	MATCH_START
 Daily Schedule	Morning schedule dispatch (08:00 AM)	send_league_daily_update	DAILY_SCHEDULE

4.4. Complete Collapse Keys Table

Event Type	Collapse Key Format	Tray Behavior
Goal	match_{id}_goal	Replaces earlier scoreline with the newest score.
Half-Time	match_{id}_ht	Replaces earlier in-play alerts with HT summary.
Full-Time	match_{id}_ft	Final match banner.
Lineups	match_{id}_lineups	Official starting XI notification.
Kickoff	match_{id}_kickoff	Match start notification.
2nd Half	match_{id}_2h	Second half resumption.
Extra Time	match_{id}_et	Extra time status.
Penalties	match_{id}_penalties	Penalty shootout status.
Status	match_{id}_status	Match disruption alert (postponed, abandoned).
Missed Pen	match_{id}_missedpen_{elapsed}	Unique per minute.
Own Goal	match_{id}_owngoal_{elapsed}	Unique per minute.

5. Device Token Management & Multi-Device Sync

5.1. Registration Flow (POST /api/notifications/devices/register/)

[Flutter Client: Startup / Token Refresh]



[POST /api/notifications/devices/register/]

Headers: Content-Type: application/json

Body: {

"registration_id": "<FCM_TOKEN>",

"type": "ios" | "android",

"language": "it",

"guest_id": "<DEVICE_UUID>" (required for anonymous users)

}



[Backend: FCMDeviceView]

- Updates registration_id, type, language, active=True
- If Language changed: calls update_device_topic_subscriptions_task
- If New Device: calls sync_device_subscriptions_task

6. Forensic Breakdown: The 3-Device Discrepancy

1. **Guest Mode Isolation:** Each unauthenticated phone generates a distinct local `guest_id` UUID. Favoriting a match on iPhone 1 modifies only iPhone 1's record in `GuestFavorite`. To sync across devices, log into the same user account or favorite on each device.
2. **Language Guard Filter:** The Flutter client silently drops notifications where the topic suffix doesn't match the current active UI language:

```
if (topicName.endsWith('_$lang') && lang != currentAppLang) {
  return false; // DROPPED SILENTLY
}
```

3. **Android 13+ & OEM Doze Mode:** Power management on Xiaomi MIUI, Samsung OneUI, and Huawei EMUI suspends Google Play push connections when locked. Delivery resumes upon screen wake-up.
4. **iOS APNs Handshake:** FCM cannot register an iOS hardware token until Apple returns `getAPNSToken()`. Network latency during cold-start can delay registration.

7. Verification & Testing Commands

```
# 1. Run full unit test suite (Docker environment - 47 tests)
docker compose exec web python manage.py test notifications sports

# 2. Run unit tests (Local virtual environment)
python manage.py test notifications sports

# 3. Direct Celery live ticker test in shell
docker compose exec web python manage.py shell -c "from sports.tasks import update_live_fixtures

# 4. Diagnostic test push via REST API
curl -X POST https://api.scorelivepro.it/api/notifications/test-push/ \
  -H "Content-Type: application/json" \
  -d '{"token": "all", "title": "⚽ Test Goal Notification", "body": "Testing multi-device synch
```